IES SAN ANDRÉS

ESPECIALIZACIÓN EN INTELIGENCIA ARTIFICIAL Y BIG DATA

SISTEMAS DE BIG DATA

UT03: GESTIÓN DE LOS DATOS

ÍNDICE

1.- Introducción a ETL con Pandas

2.- Extracción: conectando con las fuentes

3.- Transformación: el corazón del proceso

4.- Carga: almacenamiento y salida

5.- Optimización y buenas prácticas

6.- Proceso ETL en entorno gráfico: Apache NiFi

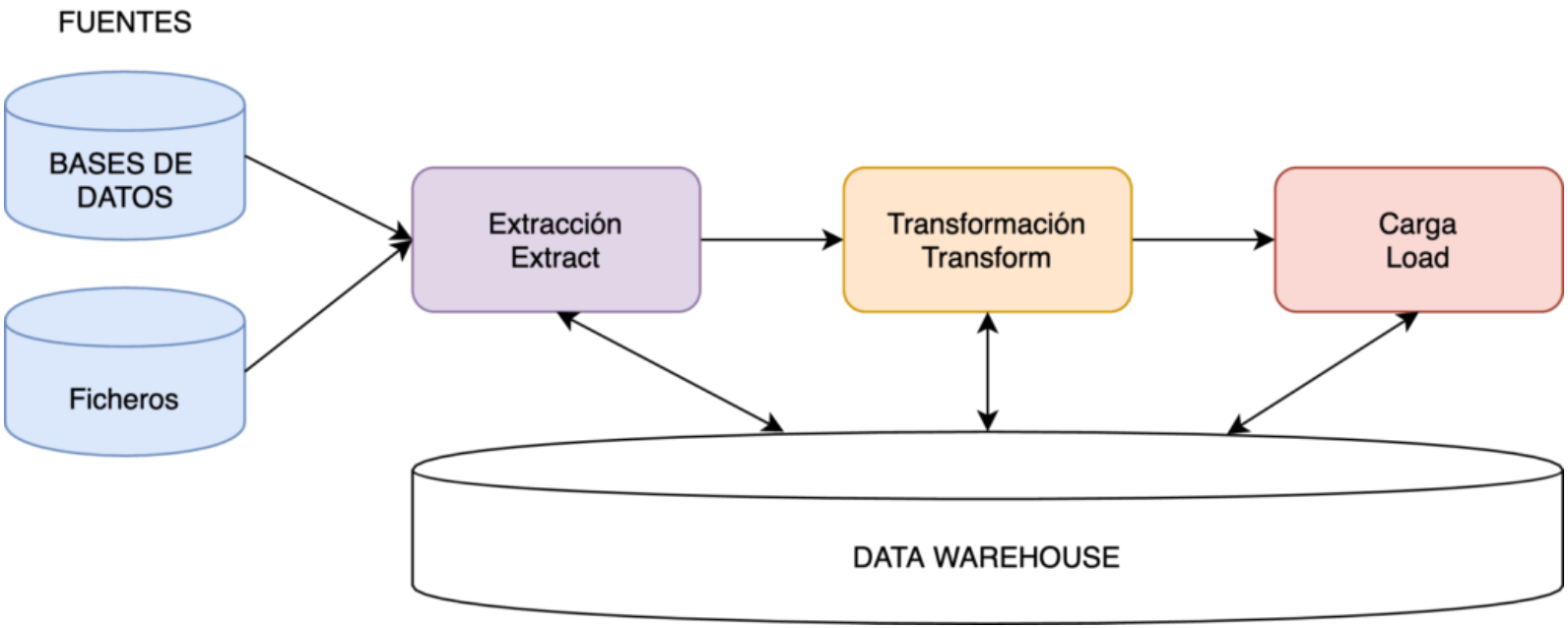
1 INTRODUCCIÓN A LOS PIPELINES DE DATOS

1 INTRODUCCIÓN A LOS PIPELINES DE DATOS

1.1 ¿QUÉ ES UN PIPELINE DE DATOS?

Un **pipeline de datos** es un conjunto de procesos estructurados que permiten **mover, transformar y almacenar datos** desde su origen hasta su destino final.

Este flujo suele automatizarse y definirse de forma secuencial.



El concepto no se refiere solo a mover los datos, sino también a **transformarlos y prepararlos** a lo largo del proceso, garantizando su calidad, integridad y utilidad.

Algunas etapas por las que pasa son:

- Limpieza
- Validación
- Enriquecimiento
- Combinación con otras fuentes

Un pipeline de datos bien diseñado:

- Permite automatizar tareas repetitivas
- Asegura trazabilidad y control de flujo de la información
- Facilita el escalado y mantenimiento de sistemas de datos
- Integra diferentes herramientas especializadas para cada fase del proceso

Un pipeline de datos se compone de varias **fases encadenadas**. Estas fases son:

- Ingesta de datos
- Procesamiento ETL y ELT
- Almacenamiento de datos
- Análisis de datos
- Consumo de datos

1. Ingesta de datos

La ingesta consiste en **capturar y recoger datos desde múltiples fuentes**, que pueden ser estructuradas (BBDD SQL), semiestructuradas (JSON, XML) o no estructuradas (log, imágenes, texto)

Ejemplos de fuentes:

- APIs (REST, SOAP)
- Bases de datos SQL y NoSQL
- Archivos planos (CSV, Excel, Parquet)
- Logs de servidores
- Sensores IoT
- Servicios de streaming (Kafka, MQTT)

La ingesta puede ser por lotes periódicos (**batch**) o en tiempo real (**streaming**)

2. Procesamiento: ETL y ELT

Esta fase se encarga de **preparar la información para su análisis y consumo**.

Hay dos enfoques diferentes:

- **ETL**: Extract, Transform, Load
- **ELT**: Extract, Load, Transform

E T L Extract – Transform - Load

Es el enfoque utilizado en arquitecturas de **Data Warehousing**

Los pasos son:

- **Extract**: se recopilan datos desde diferentes fuentes
- **Transform**: se **aplica lógica de negocio** a los datos. Por ejemplo:
  - Tareas de limpieza (eliminación de duplicados, nulos)
  - Conversión de tipos de datos
  - Normalización y estandarizaciones
  - Agregaciones y cálculos
  - Enriquecimiento
- **Load**: finalmente los datos se almacenan en un sistema destino **estructurado** como un Data Warehouse

E LT

Extract – Load - Transform

Es el enfoque utilizado en arquitecturas de **Cloud Data Warehousing** y **Data Lakes**

Los pasos son:

- **Extract**: se recopilan datos desde diferentes fuentes
- **Load**: los datos se cargan inmediatamente en el sistema destino en su estado original (*raw*) sin modificaciones previas
- **Transform**: se aplica la lógica a los datos dentro del sistema destino, por ejemplo:
  - Ejecución de consultas SQL para limpiar y filtrar datos cargados
  - Modelado de datos y creación de vistas para análisis
  - Generación de tablas agregadas para reportes finales

1 INTRODUCCIÓN A LOS PIPELINES DE DATOS

1.2 INTRODUCCIÓN A PANDAS

¿Qué es Pandas?

Pandas es una biblioteca de software de código abierto escrita para el lenguaje de programación Python.

Su propósito fundamental es la **manipulación y el análisis de datos**.

Está construido sobre **Numpy**, otra librería de Python especializada en cálculos numéricos sobre arrays de datos.

Pandas, cuyo nombre viene PANel DAta, fue desarrollado en 2008 por **Wes McKinney** mientras trabajaba en AQR Capital Management.

En 2009 fue liberado como Open Source.

En la actualidad es el **estándar de facto** en la manipulación de datos programática.

2 EXTRACCIÓN: CONECTANDO CON LAS FUENTES

2 EXTRACCIÓN: CONECTANDO CON LAS FUENTES

2.1 FORMATOS DE ARCHIVOS COMUNES: CSV

Algunos ejemplos de archivos comunes que podemos encontrar son:

- Archivos de texto plano
- Ficheros CSV (*Comma Separated Values*)
- Documentos Excel
- Archivos JSON
- Formatos específicos: Parquet y Feather

Archivos CSV

Los archivos **CSV (Comma Separated Values)** son un formato universal para el intercambio de datos.

Son simples, legibles por humanos y soportados por casi cualquier sistema informático, desde mainframes de los años 80 hasta las nubes modernas.

Aunque comúnmente los llamamos CSV, en realidad son **Archivos de Texto Delimitados**, y tienen 3 componentes clave que debemos tratar:

- El delimitador
- La codificación
- El encabezado

El delimitador | La codificación | El encabezado

Es el carácter que separa una columna de la otra.

Puede ser:

- **Coma (`,`)**: el estándar oficial del CSV
- **Punto y coma (`;`):** muy común en Europa y Latinoamérica, ya que son lugares donde se usa la coma como separador decimal (10,5)
- **Tabulador(`\t`):** se llaman archivos TSV (Tab Separated Values). Son muy seguros porque es raro que un archivo contenga una tabulación accidentalmente.
- **Pipe (`|`)**: usado a menudo en sistemas antiguos o Unix
- **Delimitadores multicarácter**: a veces, un único carácter no es seguro porque el texto mismo podría contenerlo, por lo que en ocasiones podemos encontrar secuencias de dos o más caracteres como `::` o `~!~`

- **Espacios en blanco variables**: es el clásico en archivos de logs de servidores o salidas de comando de Linux, donde hay uno o más espacios de forma que se vean visualmente alineados.

```
IP            USER    STATUS
192.168.1.1   admin   OK
10.0.0.5      guest   ERROR
```

- **Caracteres invisibles (ASCII Control Characters):** en el mundo de Hadoop, Hive y Mainframes se suelen usar caracteres que no se pueden imprimir ni ver en un editor normal. Esto garantiza al 100% que el delimitador nunca aparezca dentro del texto de un usuario. Por ejemplo, `\x01` Start of Heading (es el delimitador por defecto de Hive) o `\0` (Null byte)

El delimitador	**La codificación**	El encabezado

Es la tabla de caracteres que usa el archivo para entender letras y símbolos. La elección de un sistema de codificación incorrecto mostrará caracteres extraños (`EspaÃ©a` en lugar de `España`).

Los sistemas más habituales son:

- **UTF-8**: el estándar moderno. Soporta emojis y todos los idiomas. Siempre intenta este primero.
- **Latin-1 (ISO-8859-1)**: el estándar antiguo de Windows/Europa Occidental. Muy común en archivos generados por Excel antiguos o sistemas *legacy* bancarios

El delimitador | La codificación | **El encabezado**

Opcionalmente, la primera fila del archivo puede contener los nombres de las columnas.

Si el fichero no tiene encabezado, habrá que proporcionar los nombres manualmente durante la extracción.

pd.read_csv()

Carga un fichero CSV en un dataframe

```python
import pandas as pd

df = pd.read_csv('datos.csv')

print(df.head())
print(df.dtypes)
```

Pandas detecta automáticamente encabezados y separadores

```
     ID         Nombre      Departamento  Salario_Anual Fecha_Contratacion
0  1001     Ana García         Marketing          45000         2021-03-15
1  1002    Carlos Ruiz            Ventas          38500         2022-07-01
2  1003  Elena Vázquez        Desarrollo          52000         2020-11-20
3  1004   Jorge Méndez  Recursos Humanos          32000         2023-01-10
4  1005   Lucía Torres          Finanzas          47000         2019-05-25
ID                    int64
Nombre               object
Departamento         object
Salario_Anual         int64
Fecha_Contratacion   object
dtype: object
```

También reconoce los tipos de datos de las columnas

Algunos parámetros que puede tener son:

- **`sep`** o **`delimiter`**: el carácter que separa las columnas, por defecto coma
- **`header`**: fila que contiene los nombres de las columnas. `0` es la primera y `None` para indicar que no tiene.
- **`names`**: si `header=None` o se quieren renombrar. Es una lista con una cadena por cada campo.
- **`index_col:`** usa una columna del archivo como índice etiqueta de fila en lugar de números consecutivos. Ej.: `index_col='ID_Cliente'`
- **`use_cols:`** carga solo las columnas que se indiquen. Ahorra mucha RAM
- **`nrows:`** lee solo las N primeras filas. Útil para pruebas
- **`skiprows:`** salta filas al inicio (títulos, …)

- **`dtype`**: fuerza el tipo de dato de las columnas. Ej.: `dtype={'ID': str, 'Edad': 'int32}`
- **`parse_dates`**: intenta convertir columnas a objetos datetime automáticamente. Ej.: `parse_dates=['fecha_compra']`
- **`converters`**: aplica una función de Python a los datos mientras lee. Lento pero potente. Ej.: `converters={'precio': limpiar_moneda}`
- **`na_values`**: qué hacer si una fila tiene más columnas de las esperadas. Los posibles valores son `error` (opción por defecto), `skip` y `warn`
- **`encoding`**: codificación de caracteres, algunos valores pueden ser `utf-8`, `latín-1` o `cp1252`

Ejemplo:

```python
import pandas as pd

df = pd.read_csv(
    'datos.csv',
    sep=',',
    header=0,
    index_col='ID',
    encoding='utf-8',
    parse_dates=['Fecha_Contratacion'],
    dtype={'Departamento': 'category'}
)

print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 5 entries, 1001 to 1005
Data columns (total 4 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Nombre              5 non-null      object
 1   Departamento        5 non-null      category
 2   Salario_Anual       5 non-null      int64
 3   Fecha_Contratacion  5 non-null      datetime64[ns]
dtypes: category(1), datetime64[ns](1), int64(1), object(1)
memory usage: 377.0+ bytes
None
```

Aunque la función read_csv() funciona muy bien, **en el mundo real lo habitual es encontrar archivos mal formados o corruptos** que hacen que el script lance un error fatal a mitad de proceso.

Veamos cómo gestionar algunos de los **errores más comunes** al leer archivos de texto con Pandas.

Error de estructura (`ParseError`)

Ocurre cuando una fila tiene más o menos columnas de las que debería. Por defecto, Pandas lanzará un `ParseError` y detendrá todo.

La solución es utilizar el parámetro `on_bad_lines='warn'`, que mostrará un aviso pero no interrumpirá la carga

Error de tipo de dato

Ocurre cuando una columna numérica tiene basura textual.

La estrategia es no forzar `dtype='int'`. Es mejor leer como `object` y limpiar en el paso de transformación.

	ID	Nombre	Departamento	Salario_Anual	Fecha_Contratacion
1	1001	Ana García	Marketing	45000	2021-03-15
2	1002	Carlos Ruiz	Ventas	38500	2022-07-01
3	1003	Elena Vázquez	Desarrollo	52000	2020-11-20
4	1004	Jorge Méndez	Recursos Humanos	ERROR	2023-01-10
5	1005	Lucía Torres	Finanzas	47000	2019-05-25

La estrategia es no forzar `dtype='int'` ya que lanzará error.

```
df = pd.read_csv(
    'datos.csv',
    sep=',',
    encoding='utf-8',
    dtype={'Salario_Anual': int}
)

print(df.head())
```

Si forzamos leerlo como entero lanzará error

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
File parsers.pyx:1160, in pandas._libs.parsers.TextReader._convert_tokens()
```

Es mejor leer como `object` y limpiar en el paso de transformación.

Leemos todo como cadenas

Convertimos a valor numérico

```
df = pd.read_csv('datos.csv', dtype=str)

df['Salario_Anual'] = pd.to_numeric(df['Salario_Anual'], errors='coerce')

datos_limpios.head()
```

Las posibilidades son:
`raise`: si algo no es número lanza error
`ignore`: si algo no es número deja la columna como cadena.
`coerce`: fuerza la conversión, si algo no es número convierte a NaN

	ID	Nombre	Departamento	Salario_Anual	Fecha_Contratacion
0	1001	Ana García	Marketing	45000.0	2021-03-15
1	1002	Carlos Ruiz	Ventas	38500.0	2022-07-01
2	1003	Elena Vázquez	Desarrollo	52000.0	2020-11-20
3	1004	Jorge Méndez	Recursos Humanos	NaN	2023-01-10
4	1005	Lucía Torres	Finanzas	47000.0	2019-05-25

Por ejemplo, eliminando las columnas con NaN

```
errores = df[df['Salario_Anual'].isna()]
datos_limpios = df.dropna(subset=['Salario_Anual'])

datos_limpios.head()
```

	ID	Nombre	Departamento	Salario_Anual	Fecha_Contratacion
0	1001	Ana García	Marketing	45000.0	2021-03-15
1	1002	Carlos Ruiz	Ventas	38500.0	2022-07-01
2	1003	Elena Vázquez	Desarrollo	52000.0	2020-11-20
4	1005	Lucía Torres	Finanzas	47000.0	2019-05-25

Valores nulos no estándar

Por defecto, Pandas sabe que `NaN`, `null` o una celda vacía son nulos.

Pero podemos encontrar con datos de sistemas antiguos a veces usan cosas creativas como `?`, `-`, `SIN_DATO` o `9999`.

La solución es utilizar el parámetro `na_values` al que se le pasa una lista que define a qué se le asigna el valor NaN en el archivo.

```
df = pd.read_csv(
    'daots.csv',
    na_values=['?', 'SIN_RESPUESTA', '-', 'n/a']
)
```

El motor de lectura (engine)

A veces, el delimitador es complejo (ej: regex) o el archivo tiene caracteres muy extraños y el motor estándar de Pandas (escrito en C para velocidad) falla.

La solución es cambiar al motor de Python. Es más lento, pero mucho **más tolerante** y potente para casos extremos.

```
df = pd.read_csv(
    'archivo_raro.txt',
    sep='--',
    engine='python')
```

Por ejemplo, el motor C falla bastante con separadores multicarácter

2 EXTRACCIÓN: CONECTANDO CON LAS FUENTES

2.2 FORMATOS DE ARCHIVOS COMUNES: EXCEL

Leer archivos de Excel es diferente a leer CSV.

Mientras que CSV es texto plano simple, un archivo Excel (.xlsx) es técnicamente un contenedor comprimido (ZIP) lleno de XMLs, con estilos, múltiples hojas y metadatos.

	A	B	C	D	E	F	G
1	España						
2	Actualizado: jueves, 03 enero 2019						
3	Fecha: miércoles, 02 enero 2019						
4							
5	Estación	Provincia	Temperatura máxima (ºC)	Temperatura mínima (ºC)	Temperatura	Racha (km/h)	Velocidad máxima (km/h)
6	Estaca de Bares	A Coruña	11.1 (09:30)	9.9 (23:40)	10,5	105 (23:59)	63 (23:00)
7	As Pontes	A Coruña	11.4 (14:10)	-0.9 (04:30)	5,2		
8	A Coruña	A Coruña	12.3 (13:20)	6.0 (05:40)	9,2	37 (18:20)	24 (17:40)
9	A Coruña Aeropuerto	A Coruña	12.4 (15:50)	-2.0 (08:50)	5,2	21 (15:30)	13 (07:20)
10	Carballo, Depuradora	A Coruña	12.5 (15:30)	-2.0 (09:20)	5,3		
11	Cabo Vilan	A Coruña	11.3 (15:10)	7.2 (07:50)	9,3	54 (15:30)	43 (15:30)
12	Vimianzo	A Coruña	10.8 (15:40)	3.0 (01:00)	6,9		
13	Fisterra	A Coruña	10.9 (15:10)	5.7 (09:20)	8,3	58 (10:10)	44 (05:10)
14	Mazaricos	A Coruña	10.0 (16:10)	0.8 (04:00)	5,4		
15	Sobrado	A Coruña	10.3 (15:30)	-1.8 (03:40)	4,3		
16	Santiago de Compostela Aeropuerto	A Coruña	11.8 (15:50)	1.3 (04:00)	6,6	18 (13:10)	7 (13:20)
17	Noia	A Coruña	12.2 (16:10)	4.2 (07:50)	8,2	26 (12:20)	9 (12:40)
18	Boiro	A Coruña	15.9 (15:40)	0.9 (05:40)	8,4		
19	Padrón	A Coruña	13.8 (15:30)	-0.6 (07:40)	6,6	34 (12:20)	21 (12:20)
20	Santiago de Compostela	A Coruña	13.1 (16:10)	1.5 (07:30)	7,3	21 (20:10)	9 (01:40)

Esto hace que la extracción sea **más lenta** y requiera parámetros específicos para navegar dentro del archivo.

pd.read_excel()

Esta es la función que utilizaremos para leer archivos de Excel desde Pandas.

Pero requiere **instalar librerías** para poder hacerlo:

- Para **.xlsx** (Excel actual): `openpyxl`
- Para **.xls** (Excel 97-2003): `xlrd`

```
!pip install openpyxl
!pip install xlrd
```

```
Requirement already satisfied: openpyxl in /opt/conda/lib/python3.11/site-packages (3.1.2)
Requirement already satisfied: et-xmlfile in /opt/conda/lib/python3.11/site-packages (from
Requirement already satisfied: xlrd in /opt/conda/lib/python3.11/site-packages (2.0.1)
```

Una característica clave de Excel son las **hojas**, ya que un archivo de Excel es un libro que contiene varias hojas.

El parámetro que podemos utilizar la seleccionar la hoja a leer es **`sheet_name`**.

Si no indicamos este parámetro por defecto leerá la primera hoja.

```
# Leemos la primera hoja
df = pd.read_excel('./Aemet2019-01-02.xls')

# Leemos la hoja llamada Sevilla
df = pd.read_excel(
            './Aemet2019-01-02.xls',
            sheet_name="Sevilla")

# Leemos la tercera hoja pasando un entero
df = pd.read_excel(
            './Aemet2019-01-02.xls',
            sheet_name=2)
```

Si pasamos el parámetro **sheet_name=None**, leerá todas las hojas, pero devolverá un **diccionario de DataFrames** en lugar de un solo DataFrame.

```
dict_df = pd.read_excel(
            "Aemet2019-01-02.xls",
            sheet_name=None
)
print(type(dict_df))
```

```
<class 'dict'>
```

Normalmente los archivos Excel tienen líneas con logotipos, encabezados,.. que proporcionan información visual pero que no es relevante para extraer datos.

En estos casos podemos usar alguno de los siguientes parámetros:

- **`header`**: indica la fila en la que se encuentra el encabezado de los datos. Por ejemplo: `header=3`
- **`usecols`**: indica las columnas que hay que leer. Ejemplo: `usecols='A:F'`
- **`skiprows`**: salta las primeras filas antes de los datos. Ejemplo: `skiprows=5`
- **`skipfooter`**: salta las filas del final de la hoja. Ejemplo: `skipfooter=2`

Ejemplo de carga de datos de Excel:

```python
df = pd.read_excel(
    'nomina_rrhh.xlsx',
    sheet_name='Plantilla_Activa',
    engine='openpyxl',
    header=2,
    usecols='A:E',
    dtype={'ID_Empleado': str}
)

print(df.head())
```

Cargamos solo una hoja

Indicamos explícitamente el motor. Opcional, pero recomendado

Los títulos están en la 3ª fila

Solo cargamos las columnas A hasta la E

En este ejemplo, indico que la columna ID_Empleado se leerá como cadena

2 EXTRACCIÓN: CONECTANDO CON LAS FUENTES

2.3 FORMATOS DE ARCHIVOS COMUNES: JSON

El formato **JSON (JavasScript Object Notation)** es el estándar actual para el intercambio de datos en la web (APIs) y bases de datos NoSQL.

A diferencia de CSV o Excel, JSON es **semi-estructurado y jerárquico** (anidado), lo que supone un reto único: hay que *aplanarlo* para que quepa en una tabla.

```
{
    "count": 7,
    "items": ["socks", "pants", "shirts", "hats"],
    "manufacturer": {
        "name": "Molly's Seamstress Shop",
        "id": 39233,
        "location": {
            "address": "123 Pickleton Dr.",
            "city": "Tucson",
            "state": "AZ",
            "zip": 85705
        }
    },
    "total_price": "$393.23",
    "purchase_date": "2022-05-30",
    "country": "USA"
}
```

pd.read_json()

Con esta función podemos leer un archivo JSON

```
[
  {"id": 1, "producto": "Laptop", "precio": 1200},
  {"id": 2, "producto": "Mouse", "precio": 25}
]
```

```
# Pandas infiere que es una lista de registros
df = pd.read_json('datos.json')
print(df)
```

El problema es cuando encontramos un **JSON anidado**, es decir, tiene datos dentro de datos

```
[
  {
    "id": 1,
    "nombre": "Ana",
    "contacto": {
        "email": "ana@test.com",
        "telefono": "555-0001"
    }
  }
]
```

Si en este caso usamos read_json() obtendremos una columna llamada contacto que contiene diccionarios enteros, lo que no nos sirve.

Necesitamos **aplanar el JSON**.

pd.json_normalize()

Esta función *explota* las jerarquías

```python
import json

with open('usuarios.json') as f:
    data = json.load(f)

df = pd.json_normalize(data)

# RESULTADO VISUAL DEL DATAFRAME:
# id | nombre | contacto.email | contacto.telefono
# 1  | Ana    | ana@test.com   | 555-0001
```

Primero lo cargamos con la librería estándar json

Y luego lo aplanamos

Observa como nombra las columnas para reflejar la jerarquía del archivo JSON

Ejemplo:

```python
df = pd.read_json('./components.json')
df
```

	id	nombre	especificaciones
0	101	Laptop Gamer	{'ram': '16GB', 'disco': '1TB SSD'}
1	102	Ratón Inalámbrico	{'ram': 'N/A', 'disco': 'N/A'}

```python
with open('components.json') as f:
    data = json.load(f)

df = pd.json_normalize(data)
df
```

	id	nombre	especificaciones.ram	especificaciones.disco
0	101	Laptop Gamer	16GB	1TB SSD
1	102	Ratón Inalámbrico	N/A	N/A

```json
[
  {
    "id": 101,
    "nombre": "Laptop Gamer",
    "especificaciones": {
      "ram": "16GB",
      "disco": "1TB SSD"
    }
  },
  {
    "id": 102,
    "nombre": "Ratón Inalámbrico",
    "especificaciones": {
      "ram": "N/A",
      "disco": "N/A"
    }
  }
]
```

El problema es que, si el JSON tiene más niveles de anidamiento no funcionará como esperamos:

```
[
    {
        "ciclo": "DAM",
        "curso": "Primero",
        "estudiantes": [
            {"nombre": "Juan", "nota": 8},
            {"nombre": "Lucia", "nota": 9}
        ]
    },
    {
        "ciclo": "ASIR",
        "curso": "Segundo",
        "estudiantes": [
            {"nombre": "Pedro", "nota": 5}
        ]
    }
]
```

```
with open('students.json') as f:
    data = json.load(f)

df = pd.json_normalize(data)
df
```

	ciclo	curso	estudiantes
0	DAM	Primero	[{'nombre': 'Juan', 'nota': 8}, {'nombre': 'Lu...
1	ASIR	Segundo	[{'nombre': 'Pedro', 'nota': 5}]

Para evitar esto, deberemos utilizar los parámetros:

- **`record_path`**: sirve para indicar a Pandas de dónde queremos sacar cada fila de la tabla final

```
df = pd.json_normalize(
    data,
    record_path=['estudiantes']
)
df
```

	nombre	nota
0	Juan	8
1	Lucia	9
2	Pedro	5

Observa que en este caso cargamos los datos que hay dentro de estudiantes, pero perdemos los datos de ciclo y curso.

- **`meta`**: con este parámetro indicamos qué datos del nivel superior queremos pegar al lado de cada estudiante.

```python
df = pd.json_normalize(
    data,
    record_path=['estudiantes'],
    meta=['ciclo', 'curso']
)
df
```

	nombre	nota	ciclo	curso
0	Juan	8	DAM	Primero
1	Lucia	9	DAM	Primero
2	Pedro	5	ASIR	Segundo

En este caso conservo los datos del nodo padre

JSON Lines

Algo habitual es encontrar archivos con el formato **JSON Lines**, donde cada línea del archivo es un objeto JSON independiente en lugar de ser todo el archivo un solo objeto gigante.


```
{"id": "0", "question": "-5 *   -6", "answer": "30", "num_terms": 2, "num_digits": 1}
{"id": "1", "question": "30 * 44 =", "answer": "1320", "num_terms": 2, "num_digits": 2}
{"id": "2", "question": "621242   - 793732 = ?", "answer": "-172490", "num_terms": 2, "num_digits": 6}
{"id": "3", "question": "What is 97 + -41 + -19 - -31 + -45 * 84?", "answer": "-3712", "num_terms": 6, "num_digits": 2}
{"id": "4", "question": "( -71 + 75   ) - -1 - 36 =", "answer": "-31", "num_terms": 4, "num_digits": 2}
{"id": "5", "question": "  -( -(   -(   -5 -   8 ) * 1 ) ) - -6 - -2 = ?", "answer": "21", "num_terms": 5, "num_digits": 1}
{"id": "6", "question": "( 69 - 20 * -86 * -84 ) -   -72", "answer": "-144339", "num_terms": 5, "num_digits": 2}
{"id": "7", "question": "5 - -45", "answer": "50", "num_terms": 2, "num_digits": 2}
{"id": "8", "question": "What is   -( -( 330191 - -907543   ) +     617155 )?", "answer": "620579", "num_terms": 3, "num_digits": 6}
{"id": "9", "question": "-8 - -3 * -6 =", "answer": "-26", "num_terms": 3, "num_digits": 1}
{"id": "10", "question": "What is 89 - 6   * -31?", "answer": "275", "num_terms": 3, "num_digits": 2}
```


Para hacer más eficiente la carga de datos es necesario utilizar el parámetro `lines=True`.

```python
df = pd.read_json(
    'logs_servidor.json',
    lines=True)
```

2 EXTRACCIÓN: CONECTANDO CON LAS FUENTES

2.4 FORMATOS DE ARCHIVOS COMUNES: XML

Otro tipo de archivo que encontraremos habitualmente es **XML (Extensible Markup Languaje)**.

Al igual que pasa con JSON, XML es **jerárquico** mientras que Pandas es tabular, por lo que necesitaremos **aplanar** esa estructura.

```
  <?xml version="1.0"?>
- <birds>
  - <owl id="1201">
      <species>Bubo bubo</species>
      <name>Eagle Owl</name>
      <region>Eurasia</region>
    </owl>
  - <owl id="1202">
      <species>Strix occidentalis</species>
      <name>Spotted Owl</name>
      <region>North America</region>
    </owl>
  </birds>
```

Antes de leer un fichero XML deberemos tener instalada la librería `lxml`.

```
!pip install lxml
```

pd.read_xml()

Este es el método más sencillo.

Pandas intentará detectar automáticamente la estructura repetitiva (filas) y los datos (columnas)

```xml
<biblioteca>
    <libro categoria="ficcion">
        <titulo>El Quijote</titulo>
        <autor>Cervantes</autor>
        <precio>20.50</precio>
    </libro>
    <libro categoria="educacion">
        <titulo>Python Data Science</titulo>
        <autor>VanderPlas</autor>
        <precio>45.00</precio>
    </libro>
</biblioteca>
```

```python
df = pd.read_xml('biblioteca.xml')

df
```

	categoria	titulo	autor	precio
0	ficcion	El Quijote	Cervantes	20.5
1	educacion	Python Data Science	VanderPlas	45.0

El problema es cuando hay niveles de **anidamiento**.

```xml
<empresa>
    <departamento nombre="IT">
        <empleados>
            <empleado id="1">Juan</empleado>
            <empleado id="2">Ana</empleado>
        </empleados>
    </departamento>
    <departamento nombre="HR"> ... </departamento>
</empresa>
```

```python
df = pd.read_xml('biblioteca.xml')
df
```

	nombre	empleados	departamento
0	IT	\n	None
1	HR	None	...

En esos casos debemos utilizar el parámetro **`xpath`** para indicar dónde debe buscar Pandas los datos.

```
df = pd.read_xml(
    'data.xml',
    xpath=".//empleado"
)
df
```

Le indicamos que busque la etiqueta llamada *empleado*

	id	empleado
0	1	Juan
1	2	Ana

```xml
<tienda>
    <seccion nombre="electronica">
        <producto id="A001">
            <nombre>Laptop</nombre>
            <precio moneda="USD">800</precio>
        </producto>
    </seccion>
    <seccion nombre="ropa">
        <producto id="B005">
            <nombre>Camiseta</nombre>
            <precio moneda="EUR">20</precio>
        </producto>
    </seccion>
</tienda>
```

Hay 4 formas de indicar la ruta:

- **Rutas absolutas**: comienza por barra inclinada simple (/) e indica la ruta desde la raíz hasta el elemento

```
df = pd.read_xml(
    'data.xml',
    xpath="/tienda/seccion/producto"
)
df
```

	id	nombre	precio
0	A001	Laptop	800
1	B005	Camiseta	20

- **Rutas relativas**: empieza con doble balla (//) y le dice al sistema: *busca este elemento donde sea que esté, sin importar la profundidad*.

```
df = pd.read_xml(
    'data.xml',
    xpath="//producto"
)
df
```

	id	nombre	precio
0	A001	Laptop	800
1	B005	Camiseta	20

- **Predicados**: los corchetes funcionan como un WHERE de AQL, permiten seleccionar nodos basándonos en condiciones específicas. Se indican por corchetes. Ejemplo:

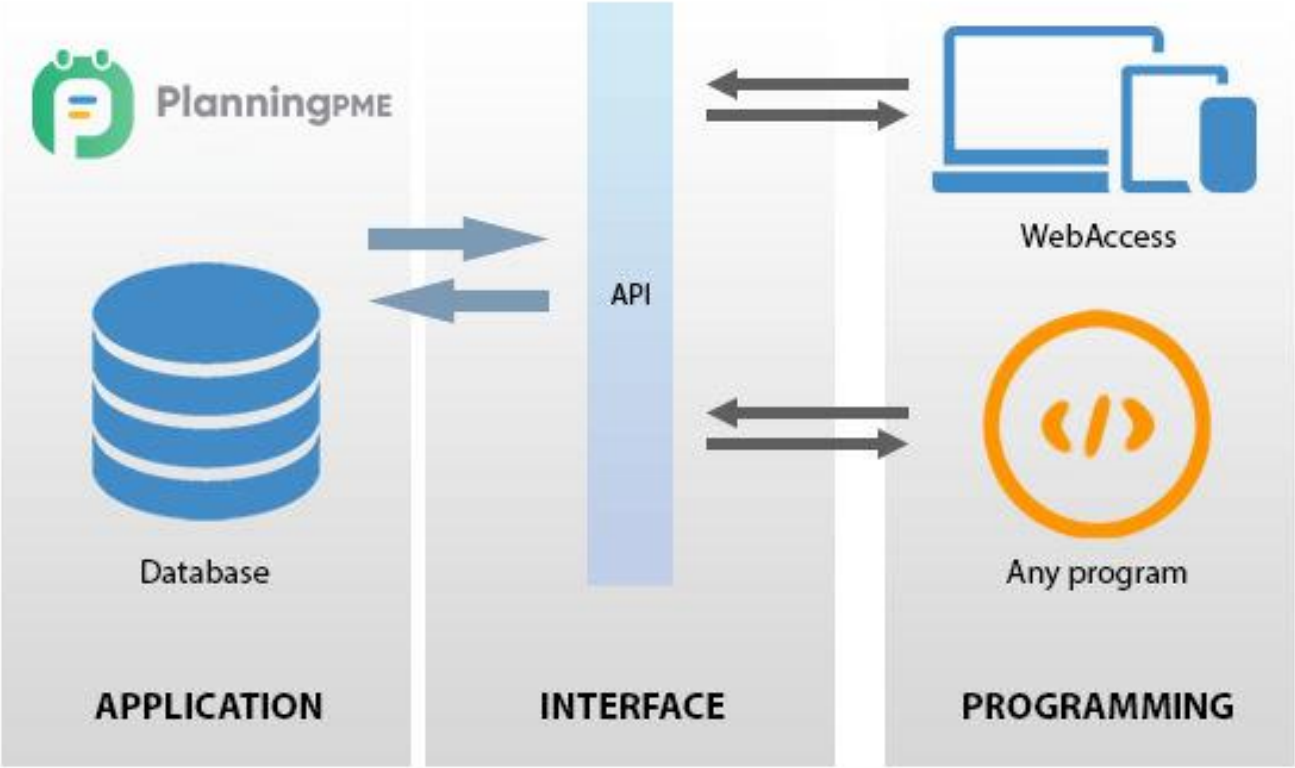
```
df = pd.read_xml(
    'data.xml',
    xpath='//seccion[@nombre="electronica"]/producto'
)
df
```

	id	nombre	precio
0	A001	Laptop	800

2 EXTRACCIÓN: CONECTANDO CON LAS FUENTES

2.5 LECTURA DE DATOS DE APIs REST

Una **API** ***(Application Programming Interface)*** es un conjunto de funcionalidades o recursos que expone un sistema para poder interactuar con él desde otro sistema, independientemente del lenguaje de programación o tecnología de cada uno de ellos.



REST (Representational State Transfer) es un estilo de arquitectura del software para comunicaciones cliente servidor apoyado en el **protocolo HTTP**.

REST se basa en tres conceptos clave:

- URLs
- Métodos HTTP
- Estados de respuesta.

URL (Uniform Resource Locator)

Una **URL** es la dirección que se le da a un recurso en la red. REST redefine este concepto utilizándolo para identificar recursos, pero también asignándoles nombres representativos.

Así, las consultas a la API son fácilmente comprensibles.

Por ejemplo:

- https://swapi.dev/api/people/1/
- https://api.twitter.com/2/users/:id/timelines/reverse_chronological

Métodos HTTP

Los métodos HTTP se utilizan para indicar qué se quiere hacer con un recurso determinado.

Se utilizan cuatro métodos principalmente, asociados con las operaciones CRUD:

- **GET**: para obtener o leer un recurso.
- **PUT**: actualiza o reemplaza un recurso
- **DELETE**: elimina un recurso del servidor
- **POST**: crea un recurso en el servidor

Estados de respuesta

El resultado de la consulta a la API se indica en el campo de estado de la respuesta HTTP.

Los estados definidos por el estándar HTTP son:

- 1xx Informational
- 2xx Success
- 3xx Redirection
- 4xx Client Error
- 5xx Server Error

Algunos ejemplos:

- **200 (OK)**: la operación solicitada se ha realizado con éxito
- **400 (Bad Request)**: código de error genérico cuando no se adapta ningún otro.
- **401 (Unauthorized)**: el usuario no ha facilitado el método de autenticación requerido por la API y no tiene acceso al recurso.
- **404 (Not Found)**: indica que la API REST no puede mapear la URI con un recurso, pero puede que sí pueda en un futuro, por lo que sí se permitirían futuras solicitudes.
- **500 (Server Error)**: código genérico para indicar algún tipo de error en el servidor.

MORE INFO https://restfulapi.net/http-status-codes/

No existe función análoga a `pd.read_csv()` que lea datos de una API.

La carga de datos de una API requiere dos pasos:

- **Extracción**: usamos la librería `requests` para obtener los datos (usualmente en formato JSON).
- **Transformación**: usamos Pandas para convertir el JSON en un DataFrame

La librería `requests`

Esta librería es el estándar de Python para interactuar con la web.

Lo primero que hay que hacer es instalarla si no la tenemos en nuestro equipo.

```
!pip install requests
```

requests.get()

Esta función nos permitirá realizar solicitudes GET a un recurso web.

```python
import requests

url = 'https://swapi.info/api/people'

response = requests.get(url)

if response.status_code == 200:
    print("¡Éxito! Conexión establecida."
    datos = response.json()
    print(datos)

else:
    print(f"Error: {response.status_code}")
```

No obtenemos los datos, sino la respuesta.

La propiedad `status_code` es fundamental para saber qué tipo de respuesta tenemos

Si el estado es 200 podemos leer los datos con `json()` o `text()`

```
¡Éxito! Conexión establecida.
[{'name': 'Luke Skywalker', 'height': '172', 'mass': '77', 'hair_cc
```

Y ya podríamos usar las funciones de JSON para importarlo a Pandas

```
pd.json_normalize(datos)
```

	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	films	species	vehicles	starships	
0	Luke Skywalker	172	77	blond	fair	blue	19BBY	male	https://swapi.info/api/planets/1	[https://swapi.info/api/films/1, https://swapi...	[]	[https://swapi.info/api/vehicles/14, https://s...	[https://swapi.info/api/starships/12, https://...	2014-12-0
1	C-3PO	167	75	n/a	gold	yellow	112BBY	n/a	https://swapi.info/api/planets/1	[https://swapi.info/api/films/1, https://swapi...	[https://swapi.info/api/species/2]	[]	[]	2014-12-1
										[https://				

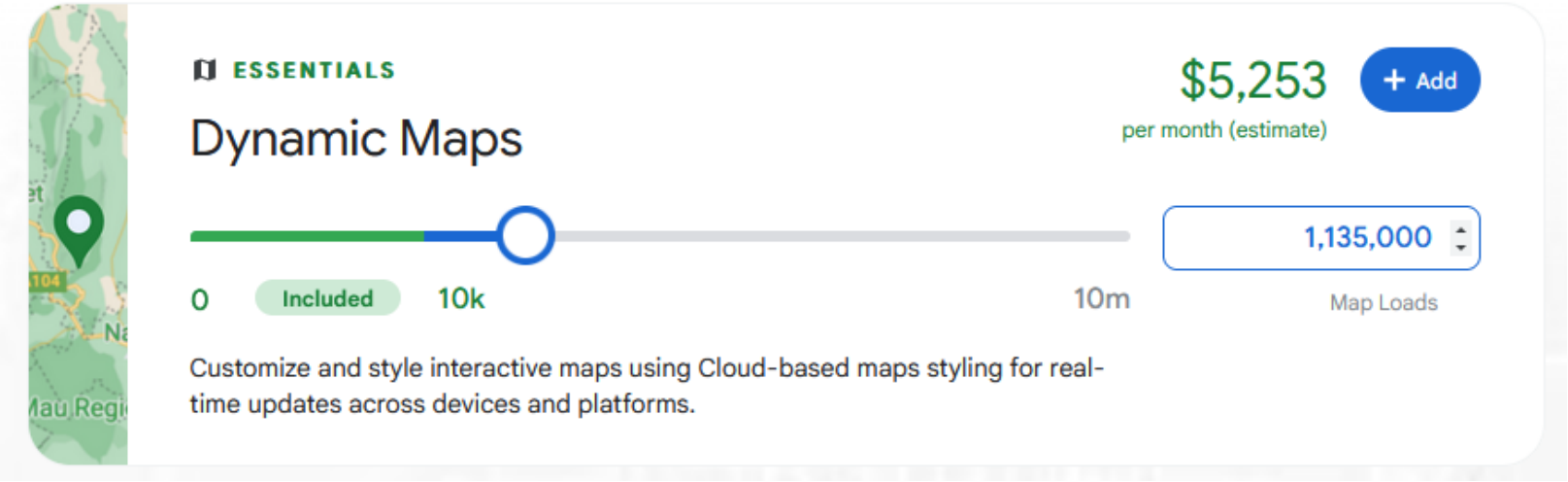
Autenticación

En el caso de una API simple con el uso del método get() es suficiente, sin embargo, lo habitual es que las APIs requieran algún tipo de autenticación, que puede ser:

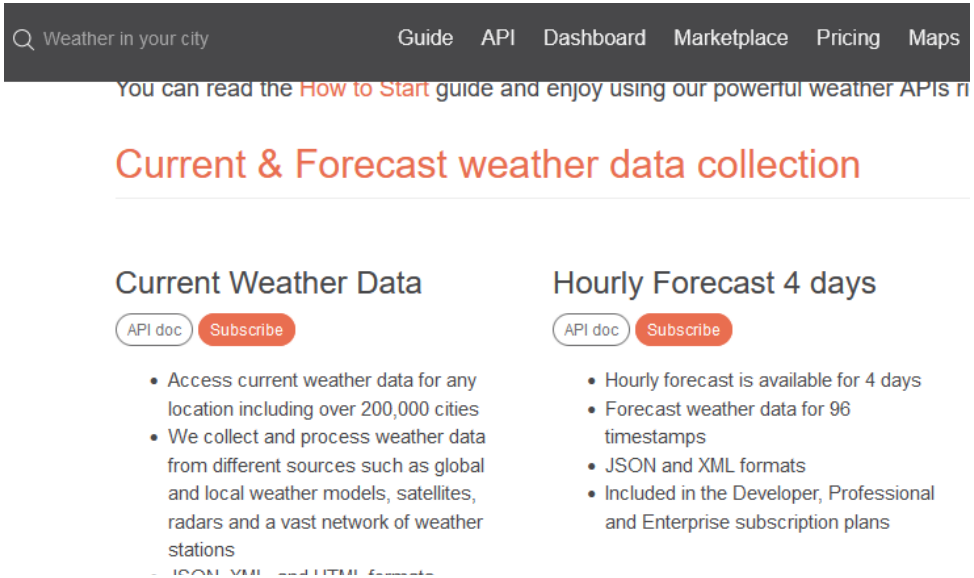
- Una **API Key** que hay que enviar en la *Query String*
- Un **Bearer Token**, que se debe enviar en la cabecera
- **Autenticación Básica**

API Key

Muchos servicios emiten una **API Key**, que básicamente es una secuencia numérica de caracteres, que **vincula nuestras solicitudes a la API con nuestra cuenta personal**, por ejemplo, para contabilizar el número de peticiones y limitarlas o cobrarlas.



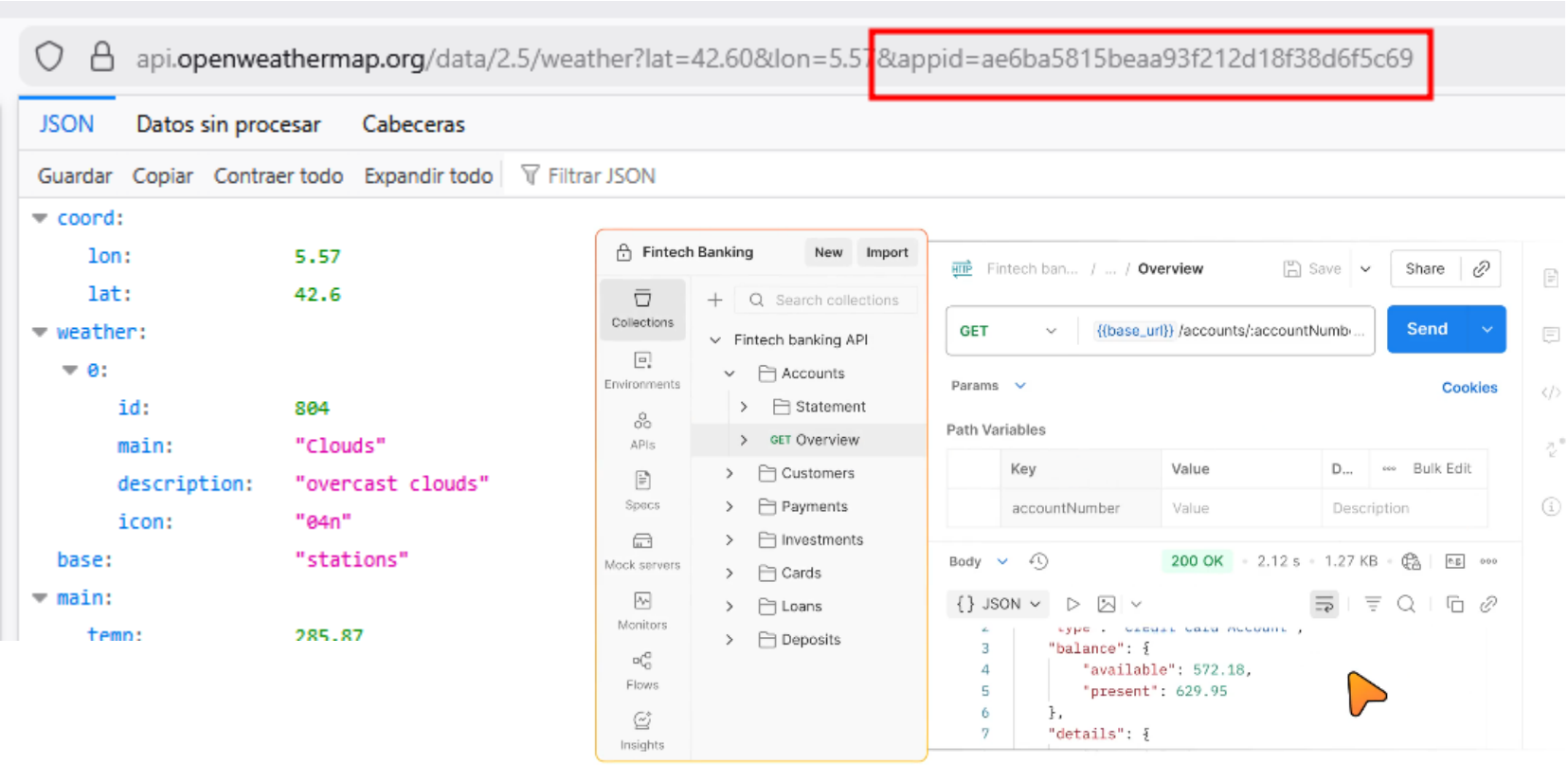
Aunque cada API especifica cómo se debe enviar la API Key, es habitual enviarla como **parámetro en la URI** en el denominado *Query String.*



API call

```
https://api.openweathermap.org/data/2.5/weather?lat={lat}&
lon={lon}&appid={API key}
```

Podemos **probar las consultas GET** escribiéndolas directamente en el navegador, aunque lo habitual si queremos hacer muchas pruebas será usar una aplicación específica como **Postman** o extensiones que ha disponibles para todos los navegadores.



Para incluir los parámetros de la *Query string* en Pandas debemos hacerlo mediante un diccionario usando el parámetro `params`.

```
params = {
    "appid": "ae6ba[redacted]",
    "lat": 42.60,
    "lon": 5.57
}
url = "https://api.openweathermap.org/data/2.5/weather"

response = requests.get(url, params=params)
response.json()
```

```
{'coord': {'lon': 5.57, 'lat': 42.6},
 'weather': [{'id': 804,
   'main': 'Clouds',
   'description': 'overcast clouds',
   'icon': '04n'}],
 'base': 'stations',
 'main': {'temp': 285.87,
```

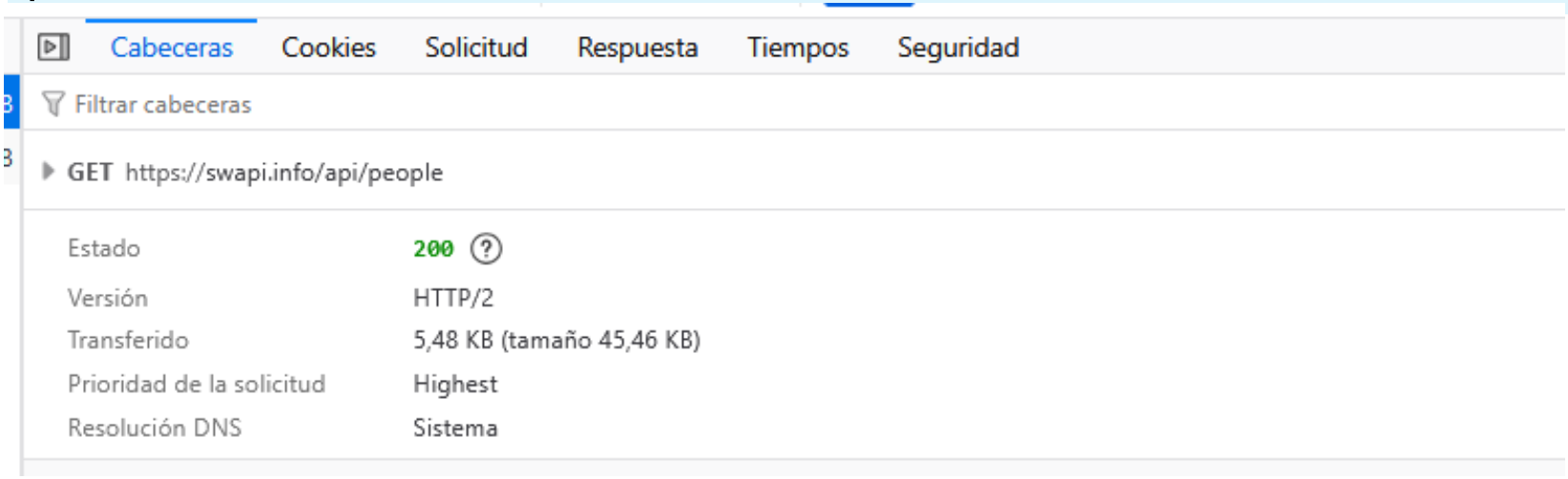
Bearer Token

El **Bearer token** es una alternativa a la API Key que se envía en el **header** del mensaje.

Características:

- Se suele usar en **OAuth**
- Al contrario que las contraseñas, los tokens suelen tener una **fecha de expiración**.
- Es **sin estado**, lo que significa que no necesita recordar que has hecho login. El servidor simplemente lee el token, verifica la firma criptográfica y te deja pasar. Esto hace que las APIs sean rápidas.

El token se envía en el header o las **cabeceras HTTP**, que son **pares nombre:valor** que se intercambian entre el navegador y el servidor en cada petición.

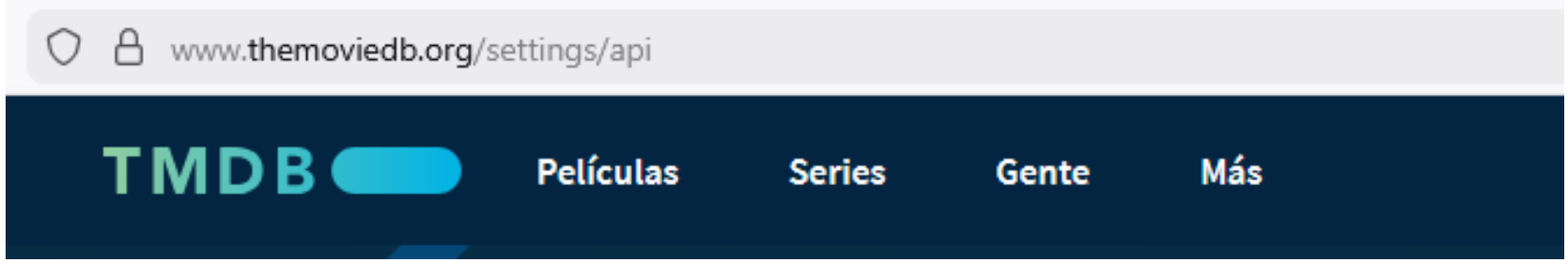


Algunas de las claves que podemos enviar en las cabeceras son:

- **`Authorization`**: con el JWT, Bearer Tokens o credenciales básicas
- **`Content-Type`**: define el formato del cuerpo (p.e. application/json)
- **`User-Agent`**: con el tipo de navegador
- **`Accept`**: fuerza al servidor a contestar en un formato determinado

Las cabeceras se pueden personalizar en las consultas con `requests` usando el parámetro **`headers`**.

Veamos un ejemplo con The Movie Database API, que usa este tipo de autenticación.



Si miramos la ayuda vemos que el token debe enviarse en el header Authorization precedido de la palabra Bearer

cURL Example

```
curl --request GET \
     --url 'https://api.themoviedb.org/3/movie/11' \
     --header 'Authorization: Bearer <<access_token>>'
```

La carga de los datos sería así:

```
BEARER_TOKEN='eyJhbGciOiJIUzI1NiJ9.eyJhdWQiOiJkNDI0ZTcwZjgwNDk4YmU0ODU0OGIyNWU2ZmE4NjljMCIsIm5iZiI
```

```
url="https://api.themoviedb.org/3/movie/popular"
headers={
    "Authorization": f"Bearer {BEARER_TOKEN}",
    "accept": "application/json"
}

response = requests.get(url, headers=headers)

data =response.json()
data
```

```
{'page': 1,
 'results': [{'adult': False,
   'backdrop_path': '/3F2EXWF1thX0BdrVaKvnm6mAhqh.jpg',
   'genre_ids': [28, 53, 80],
   'id': 1306368,
   'original_language': 'en',
   'original_title': 'The Rip',
   'overview': 'Trust frays when a team of Miami cops discovers millions in cash inside a run-dow
n stash house, calling everyone – and everything – into question.',
```

Basic Auth

Este método es habitual en sistemas antiguos (como JIRA). En estos casos debemos utilizar el parámetro **auth**

```
from requests.auth import HTTPBasicAuth

user = "mi_usuario"
password = "mi_password"

response = requests.get(url, auth=HTTPBasicAuth(user, password))
```

2 EXTRACCIÓN: CONECTANDO CON LAS FUENTES

2.6 OBTENCIÓN DE DATOS DE LA WEB

Hay ocasiones en que nos puede interesar obtener datos directamente de una página Web.

El **web scraping** es una técnica automatizada para la extracción de información de sitios web

Dificultades:

- **Contenido dinámico**: muchas webs rellenan el HTML mediante JavaScript, lo que invalida el uso de librerías como requests.
- **Fragilidad del DOM**:el scraping depende de la estructura del HTML. Si el diseño de la página cambia, el script puede dejar de funcionar.
- **Inconsistencia de datos**: el texto web es sucio, con formatos mixtos, codificaciones extrañas y campos faltantes.
- **Bloqueo de IP**: normalmente los servidores bloquean IP que envían demasiadas peticiones en poco tiempo.

- **Detección de User-Agent**: si el script se identifica como Python-urllib o Pandas, muchos servidores rechazarán la conexión (Error 403 o 401)
- **CATPCHAS**: desafíos visuales para distinguir humanos de robots.
- **Términos de servicios**: muchas webs prohíben explícitamente el scraping.
- **Robots.txt**: el archivo estándar donde los administradores web indican qué partes del sitio permiten o prohíben el acceso a bots. Ignorarlo es una mala práctica profesional

Extracción de tablas HTML. `read_html()`

Pandas dispone de la función `read_html()`, que es una herramienta de alto nivel diseñada **para localizar, parsear y convertir etiquetas `<table>` de HTML en DataFrames** de forma automática.

Como requisito, será necesario tener instalada alguna de estas librerías que usará como motor para procesar el HTML:

- **lxml**: el motor más rápido (está en C) y el que usa por defecto
- **BeautifulSoup7 / html5lib**: son más lentos, pero más tolerantes a errores

3

CARGA: ALMACENAMIENTO Y SALIDA

3 CARGA: ALMACENAMIENTO Y SALIDA

3.1 INTRODUCCIÓN A LA CARGA DE DATOS

ESTRATEGIAS DE PERSISTENCIA

Una vez que hemos obtenido los datos de diferentes fuentes y los hemos limpiado y procesado, el siguiente paso será almacenarlos para su posterior uso desde los sistemas de ML.

En este caso podemos hablar de dos enfoques:

- **Formatos de intercambio** (*human readable*)
  - Son formatos como CSV, JSON o XML.
  - Almacenan todo como cadenas de texto plano.
  - Esto es poco eficiente al ser leídos por máquinas: por ejemplo, por parsear el texto '3.141592' a flotante cuando tengo millones de registros.

- **Formatos binarios de almacenamiento** (*machine ready*)
- Son formatos como **Parquet, Avro o ORC**.
- Almacenan los datos en formato binario replicando casi exactamente cómo se estructuran en memoria RAM.
- Permiten mapeos de memoria directos y la lectura es casi instantánea.

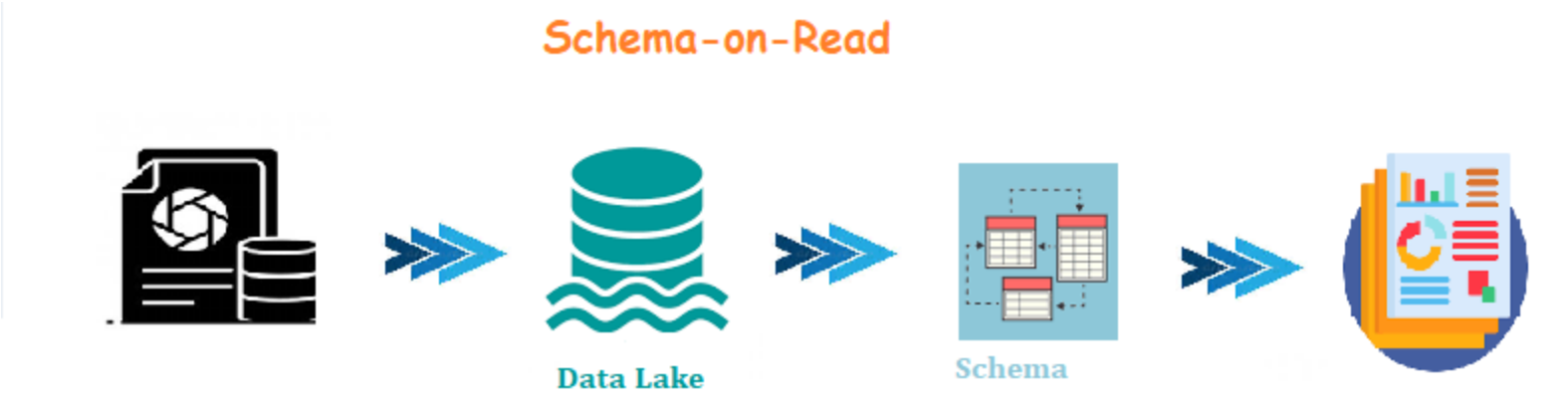
GESTIÓN DE ESQUEMAS

En bases de datos tradicionales (SQL), **el esquema se define antes de guardar**.

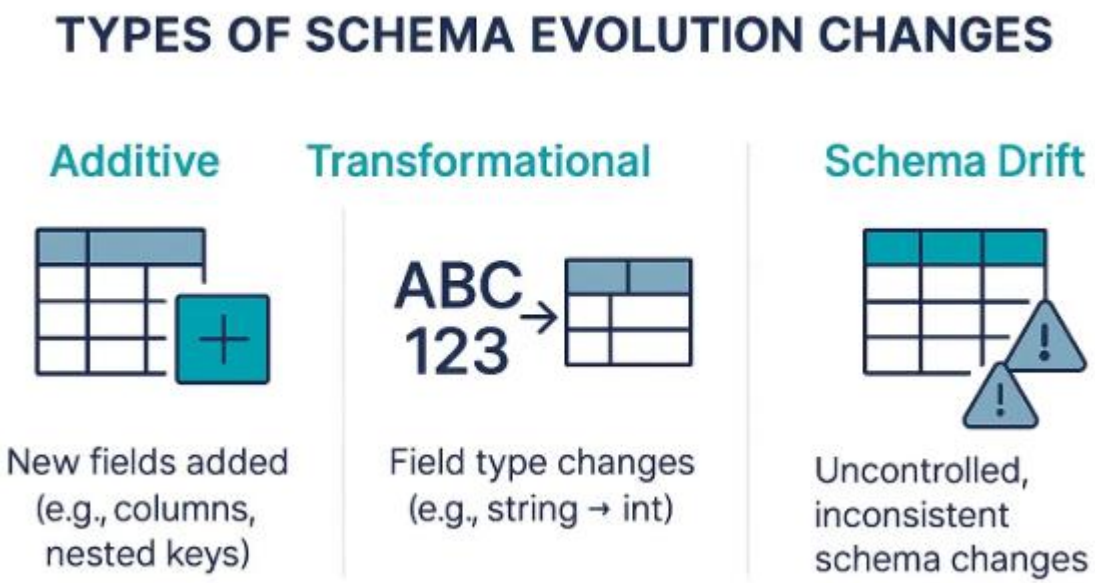
En Big Data, la estrategia cambia radicalmente:

- **Schema on write**
  - Típico de los data warehouses.
  - Los datos se validan y transforman para encajar en una tabla antes de guardarse.
  - Es rígido pero seguro

- **Schema on read**
  - Típico de *data lakes*
  - Guardamos los datos crudos (JSON, logs) tal cual llegan
  - El esquema se aplica en el momento en que un script de Python o Spark decide leerlos.
  - Da máxima flexibilidad, pero requiere mayor control.

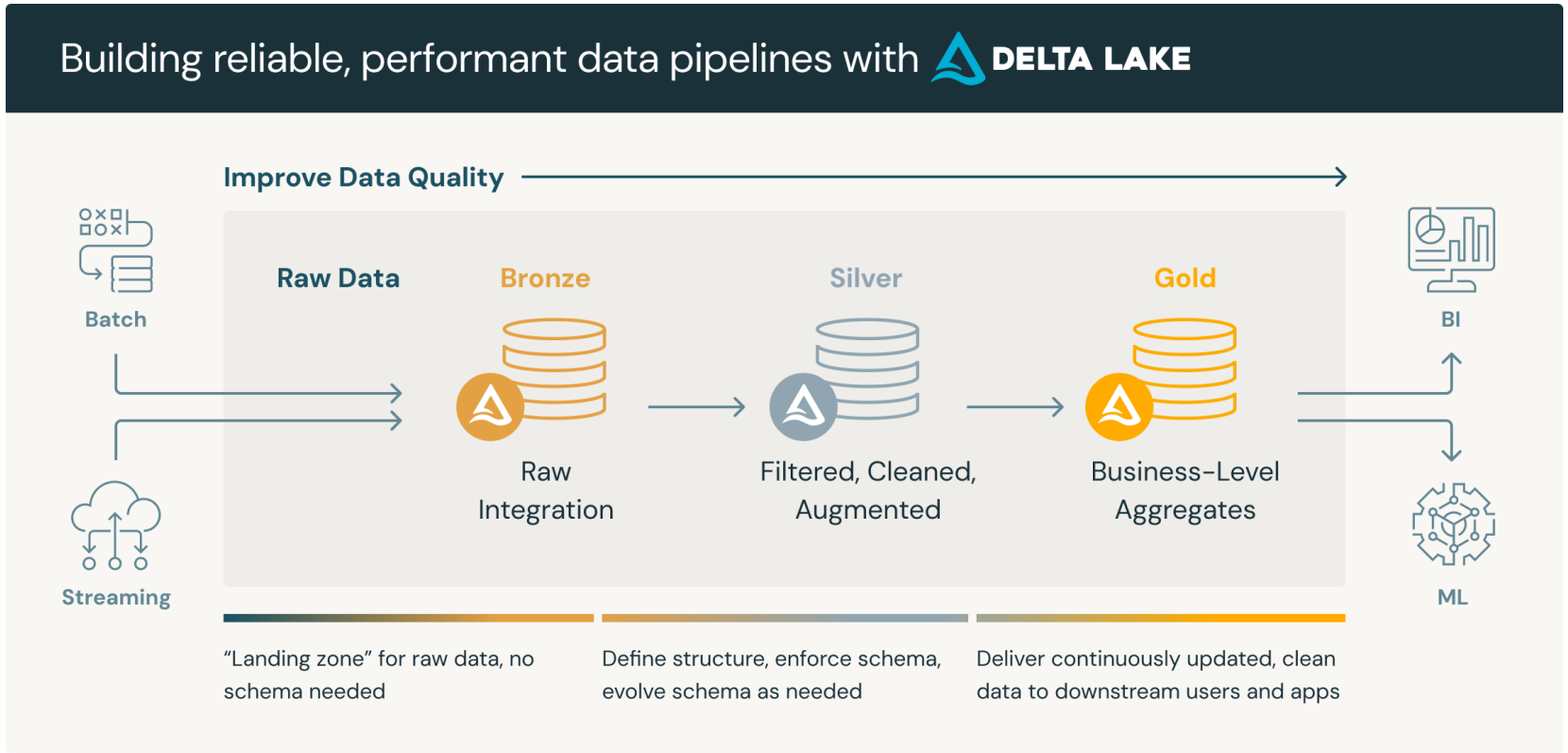


- **Evolución del esquema**
  - Estrategias avanzadas de persistencia (Delta Lake o Apache Iceberg) permiten gestionar cambios en el esquema sin corromper el histórico de datos.
  - Por ejemplo, cambiar un campo que es int a float.



ARQUITECTURA DE ALMACENAMIENTO: MODELO MEDALLÓN

En un proyecto de Big Data los datos no se guardan una sola vez. Se transforman y persisten en **diferentes capas de calidad**, un estándar conocido como **arquitectura medallón**



Se definen 3 capas:

- **Capa bronce:**
  - Los datos persisten exactamente como llegan a la fuente, a menudo en JSON o CSV
  - Si hay errores más adelante siempre podemos volver aquí
- **Capa plata**:
  - Los datos se filtran, se limpian de nulos, se estandarizan sus tipos y se persiste en un formato columnar como **Parquet**.
  - Es la fuente de verdad para analistas
- **Capa oro**
  - Datos altamente agregados o las *feature stores* (variables precalculadas listas para entrenar modelos de IA)

3 CARGA: ALMACENAMIENTO Y SALIDA

3.2 FORMATOS DE ARCHIVO BINARIOS Y COLUMNARES

Como hemos comentado, una vez preparados los datos estos se suelen almacenar en formatos binarios.

Sus características son:

- **No son legibles por humanos** mediante un editor de texto
- Están matemáticamente optimizados para ser **procesados por la CPU a altas velocidades**
- **Ocupan una fracción del espacio en disco** con respecto a los formatos de texto.

Los principales son:

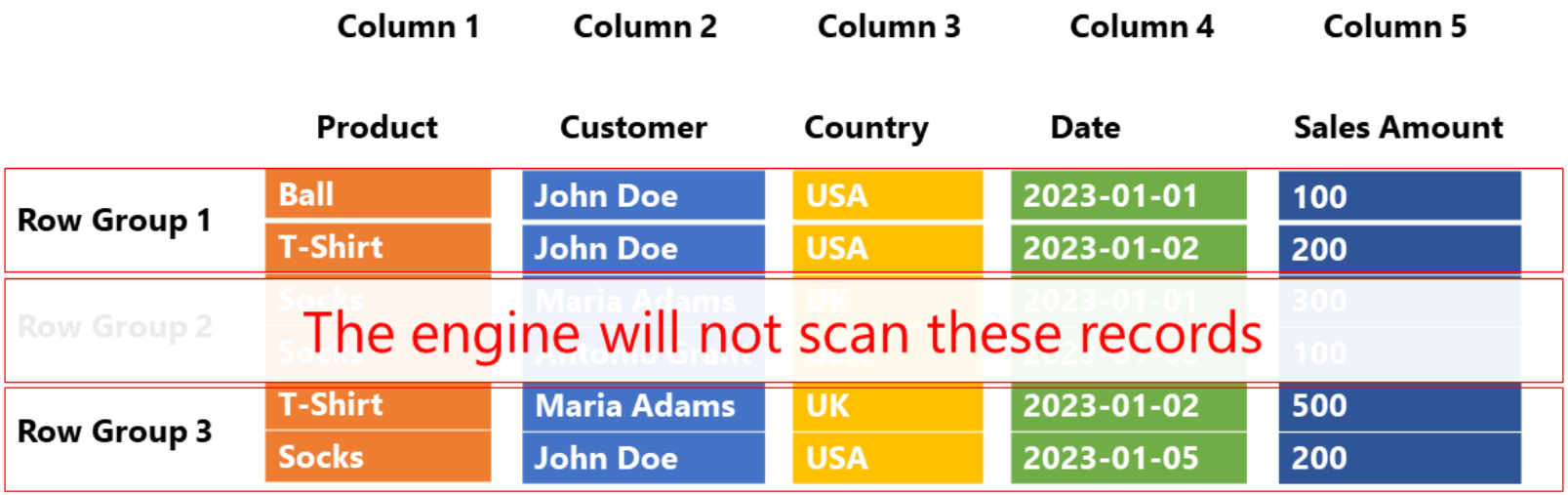
- Apache Parquet
- Apache Avro
- Apache Arrow y Feather
- HDF5

APACHE PARQUET

Apache Parquet es el estándar de facto para Data Lakes y analítica pesada

Es un formato de naturaleza **columnar**.

Los datos se guardan agrupando los valores de una misma columna, no de una misma fila.



Soporta **compresión nativa**. Al tener los datos del mismo tipo juntos (p.e. una columna llena de ceros y unos), los algoritmos de compresión (Snappy o Zstd) alcanzan ratios de reducción de tamaño de hasta 80% o 90% respecto al CSV original

El caso de uso ideal son las capas *plata* y *oro* de un Data Lake.

Uso del formato Parquet en Python

Pandas dispone del método **`to_parquet()`** para almacenar un DataFrame en un fichero en formato Parquet.

Un parámetro importante de esta función es `compression`:

- `compression='gzip'`: máxima compresión, ideal para almacenamiento en frío
- `compression=None`: sin compresión, máxima velocidad de escritura y mayor tamaño

```python
import pandas as pd
import numpy as np

# Generamos un DataFrame de ejemplo
df = pd.DataFrame({
    'id': range(1000000),
    'edad': np.random.randint(18, 99, size=1000000),
    'ciudad': np.random.choice(['Madrid', 'Barcelona', 'Valencia'], size=1000000)
})

df.to_parquet('datos_base.parquet')

df.to_parquet('datos_comprimidos.parquet', compression='gzip')

df.to_parquet('datos_rapidos.parquet', compression=None)
```

Escritura estándar (por defecto, compresión snappy y motor pyarrow)

Máxima compresión

Escritura sin compresión. Máxima velocidad de escritura y mayor tamaño

Cuando manejamos gigabytes o terabytes de datos (Amazon S3 o Hadoop), no se guarda en un único archivo gigante, sino que se dividen los datos en **directorios lógicos**.

Esto se llama **particionado hive-style**.

```
df.to_parquet(
    'datos_particionados/',
    engine='pyarrow',
    partition_cols=['ciudad']
)
```

Esto crea un fichero diferente para cada una de las ciudades. En el disco veremos algo así:

```
datos_particionados/ciudad=Madrid/part-0001.parquet
datos_particionados/ciudad=Barcelona/part-0002.parquet
datos_particionados/ciudad=Valencia/part-0003.parquet
```

Un error común al usar Parquet es leer todo el archivo cuando solo se necesitan un par de columnas.

La **proyección de columnas** permite cargar en la memoria RAM solo lo estrictamente necesario.

```
df_reducido = pd.read_parquet(
    'datos_base.parquet',
    columns=['id', 'edad']
)
```

Solo leemos las columnas id y edad

Una característica de Parquet es el **Predicate Pushdown (Filtrado en disco)** que permite filtrar los datos a medida que los leemos de disco.

```
df_filtrado = pd.read_parquet(
    'datos_base.parquet',
    engine='pyarrow',
    filters=[('edad', '>', 50)]
)

df_complejo = pd.read_parquet(
    'datos_base.parquet',
    filters=[
        ('edad', '>', 50),
        ('ciudad', '==', 'Madrid')
    ]
)
```

Solo lee los bloques que cumplen la condición. Los operadores admitidos son: `==, =, >, >=, <, <=, !=, in, not in`

Aquí hay una condición compuesta: solo se leerían las filas del archivo que cumplan ambas condiciones (operador AND)

Si queremos usar el operador OR (una condición u otro) debemos usar esta sintaxis

```
filters=[
    [('edad', '>', 50)],
    [('ciudad', '==', 'Madrid')]
]
```

APACHE AVRO

Si Parquet es para acelerar las lecturas, **Apache Avro** está diseñado para realizar **escrituras a alta velocidad**.

Es un formato **basado en filas**.

Su principal ventaja es la **evolución del esquema**: si la base de datos origen añade una columna nueva o elimina otra, Avro sabe cómo reconciliar el esquema viejo con el nuevo sin que el *pipeline* de datos se rompa.

Su caso de uso ideal son las **tuberías de datos en tiempo real** (Apache Kafka) y la **capa bronce** de los *Data Lakes*.

Son sitios donde prima la velocidad de escritura y la recolección de registros completos a medida que se generan.

Uso del formato Avro en Python

La característica más importante de Avro es que **separa el esquema de los datos**.

Los datos se guardan en binario puro, pero el archivo incluye una cabecera con el esquema escrito en formato JSON.

```python
# Definición del esquema en formato diccionario (JSON)
esquema_usuario = {
    'doc': 'Esquema para el registro de usuarios del sistema',
    'name': 'Usuario',
    'namespace': 'com.empresa.datos',
    'type': 'record',
    'fields': [
        {'name': 'id', 'type': 'int'},
        {'name': 'nombre', 'type': 'string'},
        # Un campo que puede ser nulo o entero
        {'name': 'edad', 'type': ['null', 'int']},
        {'name': 'ciudad', 'type': 'string'}
    ]
}
```

Antes de escribir los datos en Avro hay que definir un esquema en formato JSON

A diferencia de Pandas, donde se suele volcar un DataFrame entero, en Avro se suele trabajar con iteradores o listas de diccionarios (simulando eventos que llegan en tiempo real).

```python
from fastavro import writer, parse_schema

# 1. Parseamos y validamos el esquema
esquema_validado = parse_schema(esquema_usuario)

# 2. Generamos datos simulando eventos (ej. desde una API)
registros = [
    {'id': 1, 'nombre': 'Ana', 'edad': 28, 'ciudad': 'Madrid'},
    {'id': 2, 'nombre': 'Luis', 'edad': None, 'ciudad': 'Barcelona'},
    {'id': 3, 'nombre': 'Carlos', 'edad': 45, 'ciudad': 'Valencia'}
]

# 3. Escribimos al archivo binario Avro
with open('usuarios_eventos.avro', 'wb') as archivo_salida:
    writer(archivo_salida, esquema_validado, registros)

print("Datos escritos en formato Avro correctamente.")
```

Primero validamos el esquema

Y luego vamos escribiendo los datos con writer

Leer un archivo en Avro nos devuelve un **iterador de diccionarios**,

```python
from fastavro import reader
import pandas as pd

registros_leidos = []

# Abrimos y leemos el archivo
with open('usuarios_eventos.avro', 'rb') as archivo_entrada:
    lector_avro = reader(archivo_entrada)

    print("Esquema original del archivo:", lector_avro.writer_schema)

    for registro in lector_avro:
        registros_leidos.append(registro)

df_avro = pd.DataFrame(registros_leidos)
print(df_avro.head())
```

En este ejemplo guardo cada diccionario en una lista para luego cargarla en Pandas

Para definir un nuevo esquema (**evolución del esquema**) simplemente hay que decirle que lea los datos viejos con las reglas nuevas

```
esquema_nuevo = {
    'name': 'Usuario',
    'type': 'record',
    'fields': [
        {'name': 'id', 'type': 'int'},
        {'name': 'nombre', 'type': 'string'},
        {'name': 'edad', 'type': ['null', 'int']},
        # Campo nuevo con valor por defecto
        {'name': 'email', 'type': 'string', 'default': 'sin_email@empresa.com'}
    ]
}

with open('usuarios_eventos.avro', 'rb') as archivo_entrada:
    lector_avro = reader(archivo_entrada, reader_schema=parse_schema(esquema_nuevo))
```

Hemos eliminado el campo **ciudad** y añadido **email** con un valor por defecto

Indico el nuevo esquema

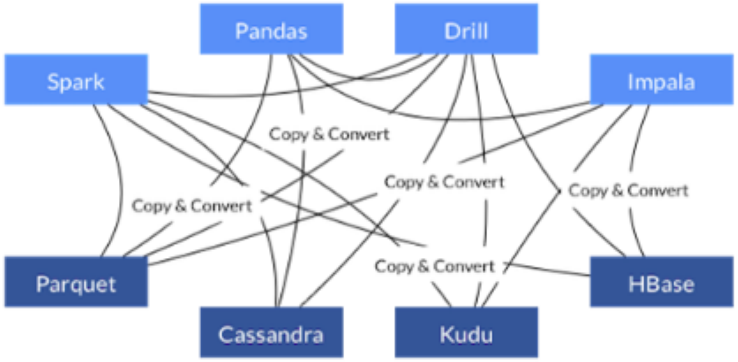
Los registros se cargarán sin la ciudad y con el correo electrónico por defecto

APACHE ARROW Y FEATHER

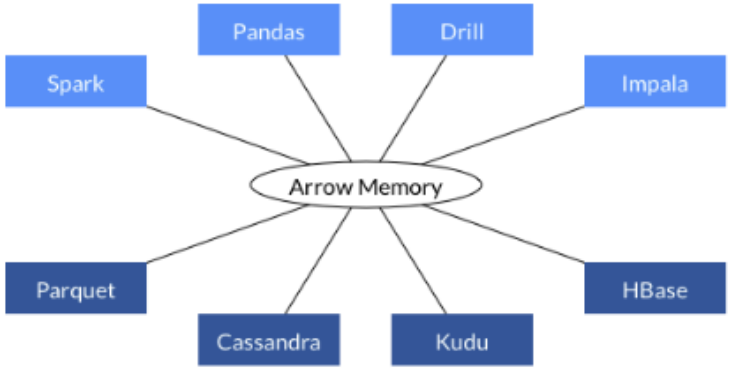
Apache Arrow no es un formato de archivo tradicional, sino **un formato columnar en memoria**.

Permite que diferentes lenguajes (Python, R, Java, …) **compartan los mismos datos en la RAM** sin necesidad de serializarlos o copiarlos.

De hecho, Pandas 2.0+ utiliza PyArrow como motor interno por defecto, sustituyendo a NumPy para muchas operaciones.



- Each system has its own internal memory format
- 70-80% computation wasted on serialization and deserialization
- Similar functionality implemented in multiple projects



- All systems utilize the same memory format
- No overhead for cross-system communication
- Projects can share functionality (eg, Parquet-to-Arrow reader)

Apache Feather es la representación en disco de Apache Arrow.

Es un formato diseñado para ser escrito y leído a la máxima velocidad que permita el disco duro.

Sus principales características son:

- **Preserva los tipos de datos** al contrario que otros tipos de datos donde todo es texto
- **Es ultra rápido**, ya que es un mapeo directo de memoria (no hay parseo de texto.
- Tiene un tamaño comprimido y eficiente

Su caso de uso ideal son los **archivos temporales.**

Por ejemplo, si necesitas guardar un DataFrame intermedio para que otro script de Python lo recoja 5 minutos después, Feather será muchísimo más rápido de leer/escribir que Parquet (aunque ocupará más espacio)

Para trabajar con Feather tenemos las funciones `to_feather()` y `read_feather()`.

```python
import pandas as pd
import pyarrow.feather as feather

df = pd.DataFrame({
    'id': range(1000000),
    'valor': [x * 1.5 for x in range(1000000)],
    'categoria': ['A', 'B', 'C', 'D'] * 250000
})

df.to_feather('datos.feather')

df_leido = pd.read_feather('datos.feather')
```

Al ser un formato columnar, puedo cargar únicamente las columnas que necesito.

```
df_parcial = pd.read_feather('datos_rapidos.feather', columns=['valor'])
```

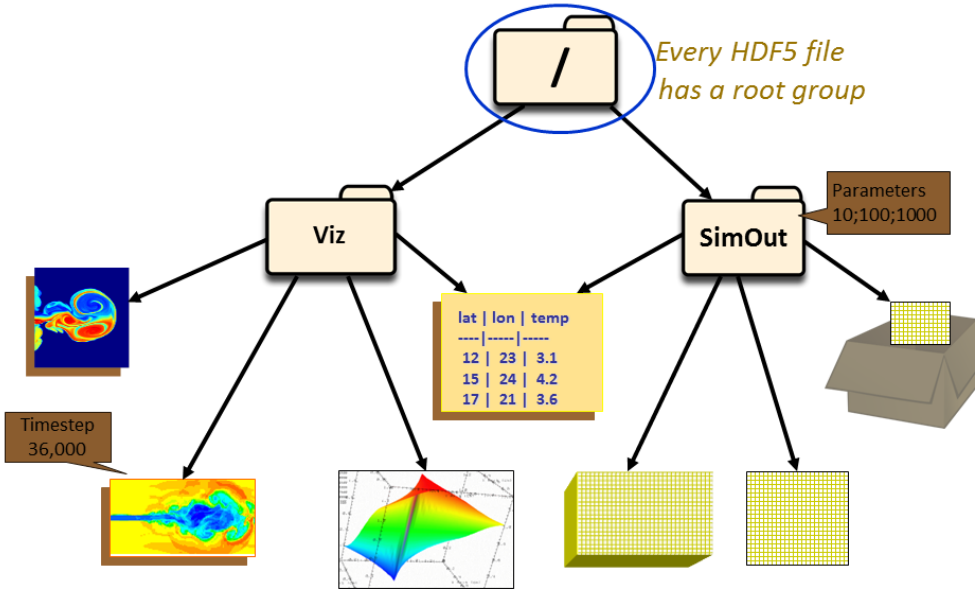
También puedo aplicar compresión a los datos al guardarlos en el fichero

```
df.to_feather('datos_comprimidos.feather', compression='zstd')
```

HDF5

El **Hierarchical Data Format (HDF5)** tiene un propósito distinto a los anteriores: no está pensado para tablas de bases de datos sino para matemáticas complejas.

Se puede decir que es un sistema de archivos dentro de un archivo. Permite **guardar matrices multidimensionales** (tensores) estructuradas **en forma de árbol** (como carpetas).



Está optimizado para lecturas parciales (slicing) de arrays gigantescos de Numpy.

Su **caso de uso ideal** es el almacenamiento de imágenes médicas, datos geoespaciales, series temporales de alta frecuencia y guardado de pesos de redes neuronales.

3 CARGA: ALMACENAMIENTO Y SALIDA

3.3 CONEXIÓN Y SALIDA A BASES DE DATOS

3 CARGA: ALMACENAMIENTO Y SALIDA

3.4 ALMACENAMIENTO EN LA NUBE

3 CARGA: ALMACENAMIENTO Y SALIDA

3.5 SERIALIZACIÓN DE OBJETOS (MODELOS IA)

3 CARGA: ALMACENAMIENTO Y SALIDA

3.6 OPTIMIZACIÓN Y COMPRESIÓN EN SALIDA